OXFORD BROOKES UNIVERSITY

Assessment cover

Module No:	COMP5047	Module title:	Applied Software Engineering

Assessment title:	Software Engineering of a Modern Computer Application

Due date and time:	23:00pm, 5th Dec. 2025

Estimated total time to be spent on assignment:	84 hours per student

LEARNING OUTCOMES

On successful completion of this assignment, students will be able to achieve the module's following learning outcomes (LOs):
1. Demonstrate an understanding of the role of requirements analysis and specification in software engineering and to be able to use this knowledge to create use case models and functional models of computer applications.
2. Demonstrate an understanding of the relationship between requirements and design and to be able to apply the knowledge to create structural and behavioural models of computer applications.
3. Critically evaluate and utilise design paradigms of object-oriented analysis and design, component-based design, and service-oriented design.
4. Use software modelling language such as UML and modelling tools in the context of model-driven software engineering.
5. Work in a group to apply the knowledge and skills developed in this module

Engineering Council AHEP4 LOs assessed	
C3	Select and apply appropriate computational and analytical techniques to model complex problems, recognising the limitations of the techniques employed
C5	Design solutions for complex problems that meet a combination of societal, user, business and customer needs as appropriate. This will involve consideration of applicable health & safety, diversity, inclusion, cultural, societal, environmental and commercial matters, codes of practice and industry standards
C6	Apply an integrated or systems approach to the solution of complex problems
C14	Discuss the role of quality management systems and continuous improvement in the context of complex problems
C16	Function effectively as an individual, and as a member or leader of a team

GROUP ID:	

STUDENT NAMES

	Student Id:	**Student Name:**	**Subsystem:**
1.	**19387617**		
2.			
3.			
4.			

Statement of Compliance *(please tick to sign)*

☐ We declare that the work submitted is my own and that the work I submit is fully in accordance with the University regulations regarding assessments (*www.brookes.ac.uk/uniregulations/current*)

RUBRIC OR EQUIVALENT:

Marking grid and marking form are available on Moodle website of the module.

FORMATIVE FEEDBACK OPPORTUNITIES

(a) Discuss your work with your practical class tutor during practical classes;
(b) Discuss your work with lecturer and/or practical class tutor in drop-in hours.

SUMMATIVE FEEDBACK DELIVERABLES

Deliverable content and standard description and criteria
A file will be in Moodle for each student to give detailed mark decomposition and additional comments as feedback on your coursework, which include:
(a) Breakdown of marks on each assessment criterion
(b) Comments on each aspect of the assessment against assessment criteria
(c) Annotations on your submitted work, if any

- **1. Introduction**

  This document presents the individual design work completed for the *Student Union Management System* (SUMS), one of the four subsystems defined in the coursework project. The purpose of the document is to provide a clear and structured explanation of the analysis and design activities carried out, following the required modelling tasks: quality requirements specification, use case modelling, activity modelling, architectural design, class design, and sequence modelling.

  The subsystem assigned — the Student Union Management System — is responsible for supporting the operations of university-level student unions, including the management and approval of society applications submitted by students. The subsystem interacts with other services such as identity management, notification services, and federation-level coordination systems.

  This report focuses on a selected functional requirement related to managing society applications, specifically the approval workflow carried out by a Union Officer. The subsequent sections document the quality requirements associated with this functionality, followed by the corresponding UML models that describe both the structural and behavioural aspects of the subsystem. The document concludes with a reflection on the individual contribution to the group work and the development process.

  - **1.1 Selected Functional Requirements**

The functional requirement selected for detailed analysis and design is "**Approve New Society**", a core capability of the **Student Union Management System**. This requirement enables a Union Officer to review, validate, and approve society applications submitted by students before they become active within the university's student union ecosystem.

It corresponds to requirement *FR-UO-3 (Management of Societies)* described in the case study, where newly proposed societies must undergo an approval workflow before being visible to students and integrated into the wider system.

- **Task 1e-2: Individual Teamwork Report**

This section provides a reflection on my individual contribution to the COMP5047 group project, covering project management, technical work, collaboration, and communication within the team.

Project Management Contribution

Throughout the project, I actively participated in the organisation and planning of group activities. During our meetings, including the weekly Tuesday sessions, I helped clarify tasks, track progress, and ensure that each member understood their responsibilities. In the first meeting, I volunteered to take the role of *Meeting Organiser*, ensuring that the group had a room booked and that all team members were informed and able to attend. I also contributed to maintaining our WhatsApp communication channel, enabling quick updates and coordination.

Additionally, I regularly checked the coursework specification to make sure that our group output aligned with the assessment criteria and deadlines. When uncertainties arose, I helped the team break down tasks so they were manageable and clearly assigned.

Technical Contribution

My primary responsibility within the group was the Student Union Management System, specifically the Society Approval workflow. I completed all required modelling tasks for this subsystem:

- I created the Use Case Diagram representing the Union Officer's interactions with society approval operations.

- I produced the Activity Diagram detailing the approval process, including decision nodes, alternative flows, and the interaction with “Request Additional Information”.

- I developed the Component Diagram reflecting how the SocietyApprovalService interacts with supporting services such as SocietyRegistry, NotificationAdapter, and AuditLogger.

- I designed the Class Diagram, defining entities such as SocietyApplication, SocietyApprovalService, Officer, and supporting components. Although Papyrus did not visually display all operations within the class boxes, I ensured that the underlying model included all required attributes and methods.

- I produced the Sequence Diagram for the approveApplication() operation, showing the step-by-step collaboration between service components.

These contributions formed the core of the subsystem’s detailed design and ensured consistency across structural and behavioural models.

Collaboration and Communication

I collaborated closely with my teammates — Harsimranjit, Sharon, and Aadam — by attending all scheduled meetings, contributing to discussions, and providing support when others needed clarification on modelling tools or coursework expectations. When multiple members were unsure about aspects of their diagrams, I shared findings from my own work in Papyrus and helped troubleshoot common issues, such as interface connections, diagram organisation, and use of modelling conventions.

We maintained effective communication throughout the project, primarily through WhatsApp and in-person meetings. I actively contributed to discussions about how individual subsystems integrate into the wider architecture, ensuring that my design remained consistent with the group’s shared vision.

Reflection

Overall, I believe my contribution has been consistent and meaningful. I managed my responsibilities effectively, produced all required deliverables for my subsystem, and supported the team both technically and organisationally. The project allowed me to deepen my understanding of UML modelling, architectural design, and collaborative software development.

- **Task 2: Analysis and Specify Software Quality Requirements**

Functional Requirement Description

The Student Union Management System enables Union Officers to review and approve new society applications submitted by students. This corresponds to the case study requirement FR-UO-3 (Management of Societies), which states that student unions must validate and approve newly proposed societies before they become active within the system.

A society application includes:

• Society name
• Theme/description
• Organisation team (leader details, student ID, USU user ID, supporting members)
• Contact information
• Optional supporting materials

Functional workflow summary

1. A student submits a new society application.
2. The Union Officer accesses the list of pending applications.
3. The system displays full registration details.
4. The Union Officer evaluates and selects either Approve or Reject.
5. If approved:

    o The society becomes active.
    o Society information is stored and made visible on the union's webpage.
    o Students can subscribe to the society.
    o A notification is sent to the applicant.

6. If rejected:

o The application is recorded as rejected.
o A notification is sent to the applicant.

This functionality interacts with other subsystems (Student App, Society Leader App, USU Operation System) through shared backend services.

Software Quality Requirements for "Approve New Society"

The following quality requirements define how the system must perform, protect, and scale when executing this functionality. All requirements are measurable, verifiable, and relevant to the selected functional requirement.

- **2.1 Security and Privacy Requirements**

S1 – Role-based Access Control

Only authenticated users with the role Union Officer must be permitted to access the "Approve New Society" interface or to modify the status of society applications.

Verification:

Attempting approval with any other role (student, society leader, USU officer) must result in a 403 Forbidden response.

S2 – Multi-Factor Authentication

Union Officers must authenticate using two-factor authentication (2FA) before accessing the approval dashboard.

Verification:

A 2FA token must successfully validate before the system exposes any pending applications.

S3 – Data Minimisation & Privacy Compliance

The system must ensure that only the necessary personal information about the society leader (name, student ID, contact details) is displayed to the Union Officer. Sensitive personal data must not be shown or logged unnecessarily.

Verification:

Data fields displayed must match the official application structure; no additional user information may appear.

S4 – Secure Audit Logging

Every approval or rejection action must be logged with timestamp, officer ID, decision, and application ID. Logs must be immutable and accessible only to authorised administrators.

Verification:

Audit logs should show an entry for every decision and must be protected from modification.

- **2.2 Performance Requirements**

P1 – Response Time

The system must display the full details of a society application within 2 seconds under normal load (≤ 50 simultaneous officer requests).

Verification:

Load testing with 50 concurrent officer sessions must result in ≤ 2s median response time.

P2 – Approval Processing Time

Once an officer clicks “Approve”, the system must complete database updates and trigger notifications within 1 second.

Verification:

The approval API endpoint should return success within 1 second in at least 95% of tests.

P3 – Dashboard Loading Efficiency

The list of pending society applications must appear within 1.5 seconds, even when the queue contains up to 500 applications.

Verification:

Test dataset of 500 entries must load ≤ 1500 ms.

- **2.3 Reliability Requirements**

R1 – Transaction Atomicity

The approval process must operate as an atomic transaction:

• Either all steps succeed (activation + visibility + notification)
• Or none of them are applied.

Verification:

Simulating a notification failure must automatically roll back the approval state.

R2 – Availability

The approval service must maintain 99.5% uptime during working hours (08:00–20:00).

Verification:

Monthly system uptime logs must show ≤ 0.5% downtime.

R3 – Data Consistency

After an approval action, the society's state must be consistent across all subsystems (backend, union webpage, Student App queries) within 5 seconds.

Verification:

Polling subsystems after approval must return consistent "active" status within 5s.

R4 – Failure Recovery

If the approval service encounters an unexpected failure, the system must automatically retry the transaction up to 3 times or revert safely.

Verification:

During stress tests, failures must trigger retries or clean rollback.

- **2.4 Scalability Requirements**

SC1 – Horizontal Scaling

The approval service must support automatic horizontal scaling to handle peak periods where up to 200 union officers nationwide may review applications simultaneously.

Verification:

Autoscaling rules based on CPU/latency must spin up additional nodes when thresholds are reached.

SC2 – Data Growth Handling

The system must maintain stable performance even if the number of society applications grows to 50,000 stored records over several academic years.

Verification:

Database query times for retrieving an application must remain ≤ 2 seconds at this scale.

SC3 – Notification Throughput

The notification subsystem must support sending up to 500 approval/rejection notifications per minute without degradation.

Verification:

Stress tests with 500 messages/min must show < 5% failure rate.

SC4 – Distributed Subsystem Consistency

When multiple universities operate simultaneously, the approval decision must propagate to all relevant subsystems within 10 seconds, regardless of load.

Verification:

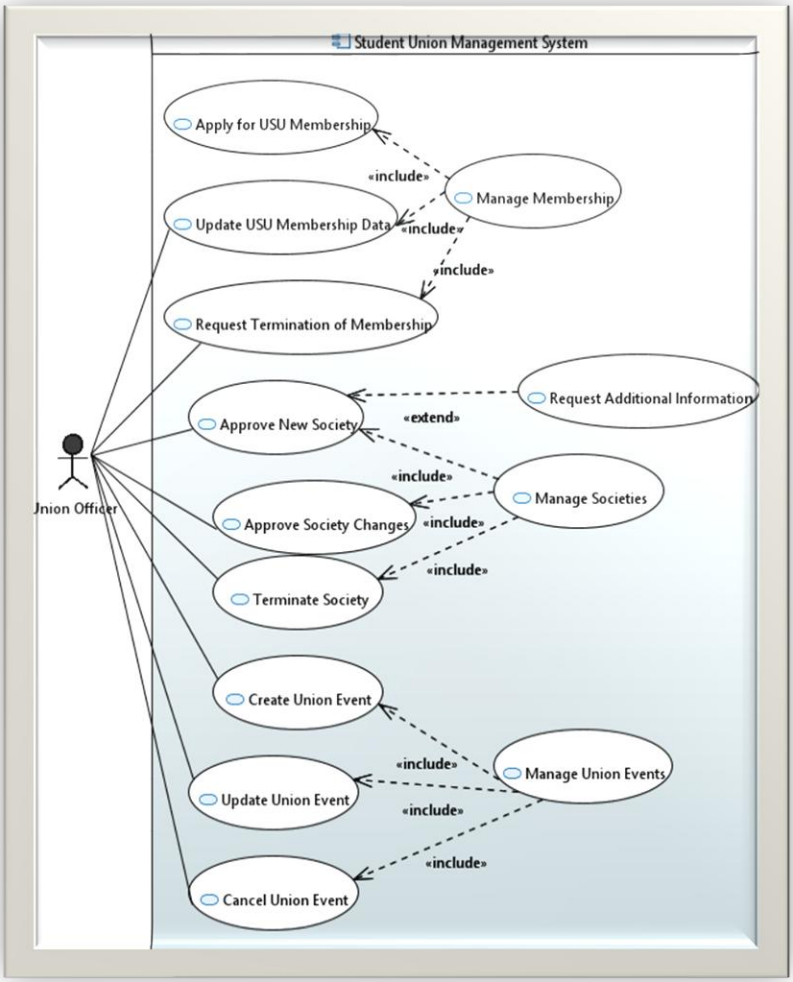
Distributed event logs must show propagation ≤ 10 seconds under simulated national usage.

Conclusion

These software quality requirements ensure that the “Approve New Society” functionality operates securely, efficiently, reliably, and at scale within the United Student Unions ecosystem.

Each requirement is measurable and testable, supporting both engineering design and future implementation within a cloud-based microservices architecture.

- **Task 3: Specification and Modelling Software Functional Requirements**
  - **Task 3A: Use Case Model**



Description

The Use Case Diagram above represents the functional scope of the Student Union Management System (SUMS) from the perspective of the *Union Officer*, the primary actor responsible for managing memberships, societies, and union events within the system.

The model groups the operations of the subsystem into three main areas:

1. **Membership Management**
   - Apply for USU Membership
   - Update Membership Data
   - Request Membership Termination
     These use cases share common behaviour captured in the abstract use case *Manage Membership*, which is included by each specialised operation to avoid duplication.

2. **Society Management**

    - Approve New Society
    - Approve Society Changes
    - Terminate Society

    These use cases include shared behaviours represented in the *Manage Societies* use case.

    Additionally, the *Approve New Society* use case uses an «extend» relationship to *Request Additional Information*, representing an optional alternative flow when the submitted information is incomplete.

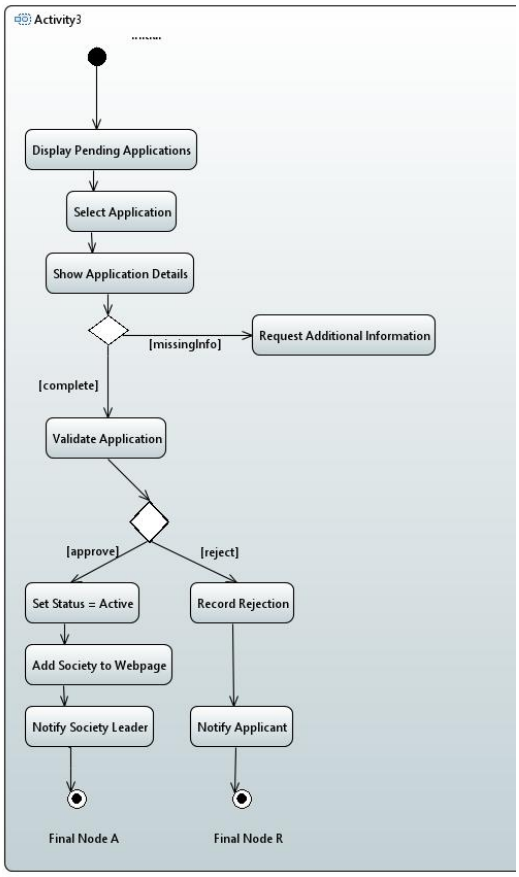
3. **Union Event Management**

    - Create Union Event
    - Update Union Event
    - Cancel Union Event

    These use cases include the common behaviour defined in *Manage Union Events*.

This diagram establishes the interaction boundaries of the subsystem and provides a high-level overview of the processes later refined in the activity, class, and sequence models.

- **Task 3B: Activity Model for "Approve New Society"**



Description

The Activity Diagram models the internal workflow followed by a Union Officer when approving a new society application. The process begins by retrieving and displaying all pending applications. The officer selects a specific application, after which the system presents the full application details.

A first decision point checks whether the submitted information is complete. If required data is missing, the process branches to *Request Additional Information* and terminates. Otherwise, the system proceeds to validate the application contents.

A second decision point determines the outcome of the validation:

- **[approve]**

The system sets the society status to *Active*, publishes it on the student union webpage, and sends a notification to the society leader before reaching the approval end node.

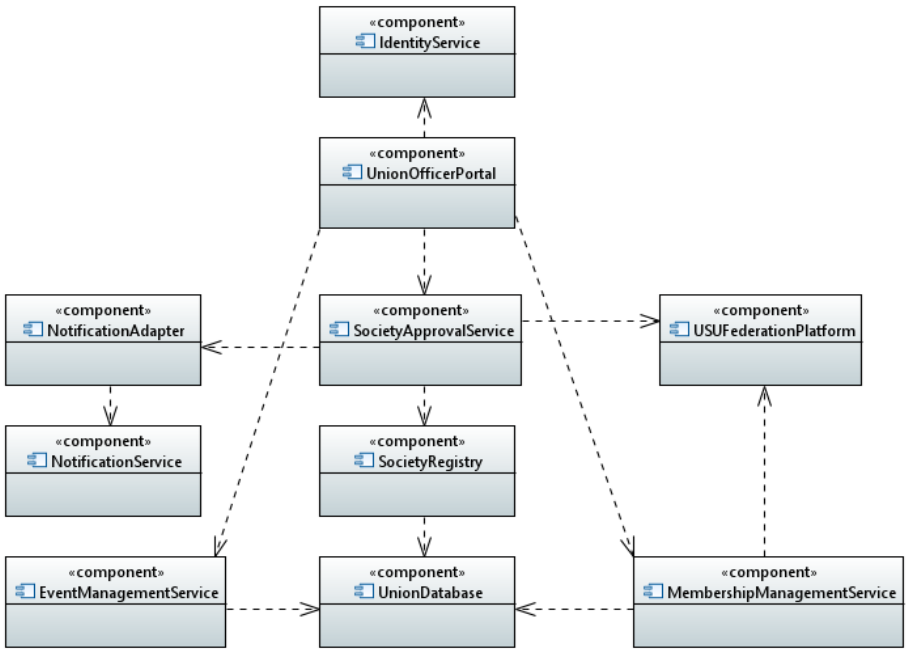
- **[reject]**

The system records the rejection and notifies the applicant, leading to the rejection end node.

This diagram clarifies the operational logic behind the ***Approve New Society*** use case and defines the foundation for the subsequent detailed design in the sequence diagram.

- **Task 4: Software Architectural Design**

  - **Task 4A: Architectural Design of the Subsystem**



Description

The Component Diagram above shows the architecture of the **Student Union Management System (SUMS)** and how it collaborates with other services in the wider USU environment. The design follows a service-oriented style, where each component encapsulates a specific responsibility and communicates with others through well-defined interfaces.

At the top, the **IdentityService** and the **UnionOfficerPortal** represent the entry point for Union Officers. The portal relies on the IdentityService to authenticate officers before granting access to the internal functions of SUMS.

The **UnionOfficerPortal** then interacts with the central backend component, **SocietyApprovalService**, which coordinates the workflow for managing societies: reviewing new applications, approving or rejecting them, and updating society records. This service does not work in isolation; it depends on several supporting components:

- **SocietyRegistry** and **UnionDatabase** provide access to persistent data. SocietyRegistry manages the catalogue of existing societies, while UnionDatabase stores membership and event information shared across the union.

- **NotificationAdapter** acts as an intermediary between SUMS and the external **NotificationService**.

When a decision is taken on a society application or an event is updated, the adapter

forwards the appropriate messages to the NotificationService, which delivers them to applicants, society leaders, or members.

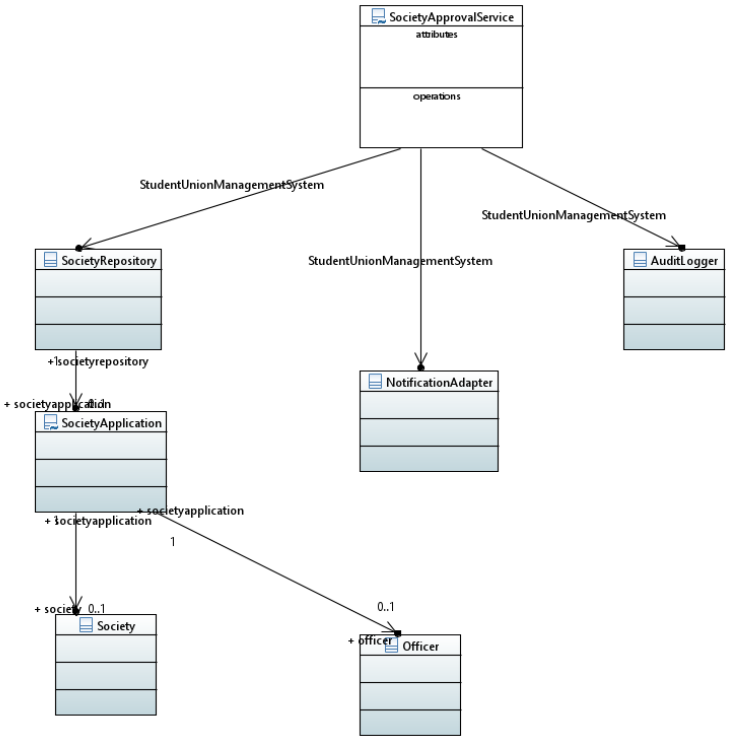
- The **EventManagementService** uses data from the UnionDatabase to handle the lifecycle of union events. It can be triggered indirectly when society approvals result in new events or updates that must be reflected in the system.

On the right-hand side, **USUFederationPlatform** and **MembershipManagementService** represent federation-level services. SocietyApprovalService sends society-related updates to the USUFederationPlatform, which in turn collaborates with MembershipManagementService to keep membership information consistent across universities. Dashed dependencies in the diagram indicate service-to-service communication rather than direct implementation containment.

Overall, this architecture cleanly separates concerns between user interaction (UnionOfficerPortal), domain logic (SocietyApprovalService), persistence (SocietyRegistry and UnionDatabase), communication (NotificationAdapter and NotificationService), event handling (EventManagementService), and federation-level coordination (USUFederationPlatform and MembershipManagementService). This modularity supports scalability and makes it easier to evolve or replace individual services without impacting the whole system.

- **Task 5: Software Detailed Design**

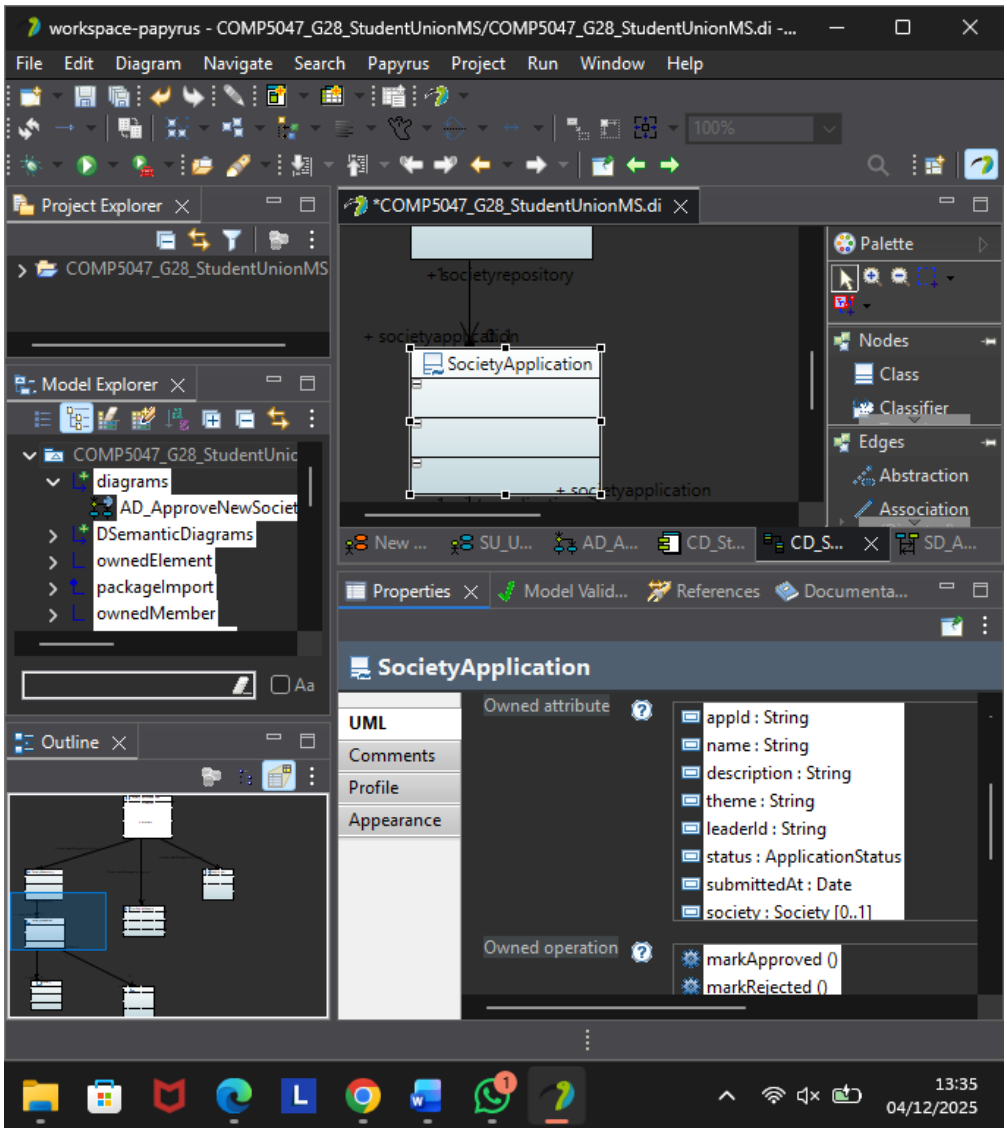
  - **Task 5A: Structural Model**



Description

The Class Diagram represents the structural design of the *Student Union Management System*, focusing on the core classes involved in the approval workflow. The central class is SocietyApplication, which models the state and properties of a

society proposal submitted by students. This class includes key attributes such as appId, name, description, theme, leaderId, status, and relevant timestamps, as well as the association to the Society entity.
Although the diagram view in Papyrus does not visually display the operations and attributes inside the class boxes, this appears to be a rendering limitation of the tool rather than an omission in the model. The internal details are correctly defined within the UML model, as demonstrated in the screenshot below, which shows the complete list of attributes and operations stored in the class definition.



The **SocietyApplication** class also defines two domain methods:

- markApproved() — updates the application status when the society is approved.
- markRejected() — sets the application as rejected and records the reason.

These methods are correctly defined in the model repository even if they are not visible within the diagram canvas.

Surrounding this class, the diagram also includes supporting components such as:

- **SocietyApprovalService**, which orchestrates the approval workflow.

- **SocietyRepository**, responsible for persisting application data.

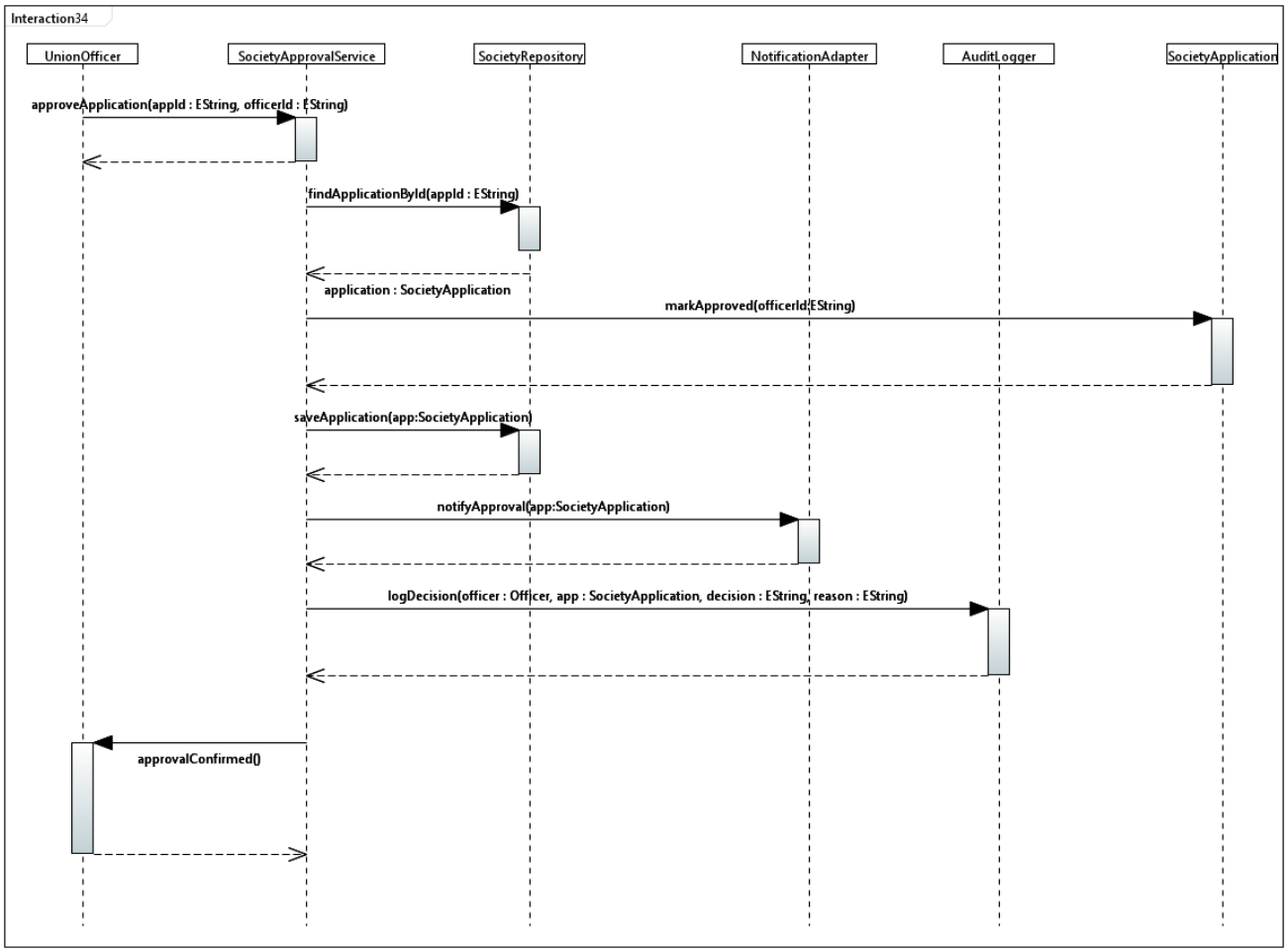
- **NotificationAdapter**, which handles communication with external notification systems.

- **AuditLogger**, which records approval or rejection decisions for traceability.

- **Officer** and **Society**, representing actors and domain entities referenced by the service operations.

Together, these classes form the structural backbone of the approval process and are consistent with the behavioural models presented in the sequence diagram.

- **Task 5B: Behaviour Model**



Description

The Sequence Diagram provides a detailed behavioural specification of the approveApplication(appId, officerId) use case. It describes how the components and objects of the subsystem collaborate to process a society application submitted by a student and issue an approval decision.
The interaction starts when the Union Officer invokes the approval operation on the SocietyApprovalService, passing the application identifier and officer identifier as parameters. The service begins by retrieving the corresponding application from persistent storage by calling findApplicationById on the SocietyRepository, which returns a SocietyApplication object.

Once the application is obtained, the service delegates domain validation to the application object itself by invoking markApproved(officerId). This method updates the internal state of the application to reflect the new status and logs the officer responsible for the decision.

After modifying the application, the service persists the new state by calling saveApplication(app) back on the SocietyRepository. The service then proceeds to notify external stakeholders: it calls notifyApproval(app) on the NotificationAdapter, which is responsible for forwarding the message to the appropriate students and society leaders.

To ensure traceability and compliance, the service records the officer’s action by calling logDecision(officer, app, "approve", "") on the AuditLogger, which stores an auditable record of the approval.

Finally, the service returns a confirmation message to the Union Officer, completing the approval flow.

This sequence model aligns with the class diagram and the system architecture, ensuring consistency between structural and behavioural design. It also reflects a clear separation of concerns: persistence is handled by the repository, communication by the adapter, logging by the audit component, and business rules by the approval service.